

Inteligência Computacional

Aula 16: Rede Neural Feed Forward – parte 2

Prof. Eduardo Corrêa Gonçalves

23/10/2026

Sumário

Introdução

Treinamento do Perceptron

- Soma do Quadrado dos Erros

- Gradiente Descendente – Intuição e Cálculo

- Exemplo passo a passo

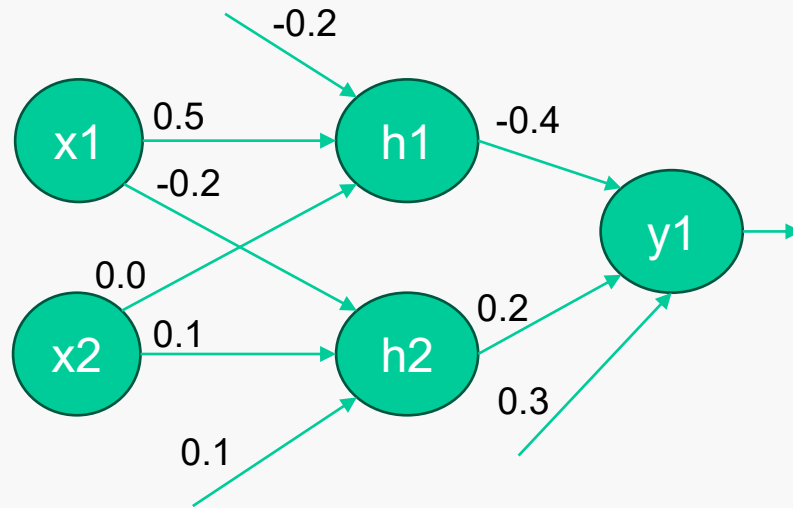
Treinamento da Rede Neural Feed Forward

- Algoritmo da Retropropagação – Intuição

- Exemplo passo a passo

Introdução (1/3)

- Relembrando as últimas aulas
 - A aula anterior mostrou como funciona o processo de **classificação** de novos objetos em uma **RNFF já treinada**.



W	
0.5	0.0
-0.2	0.1

$b^{[1]}$
-0.2
0.1

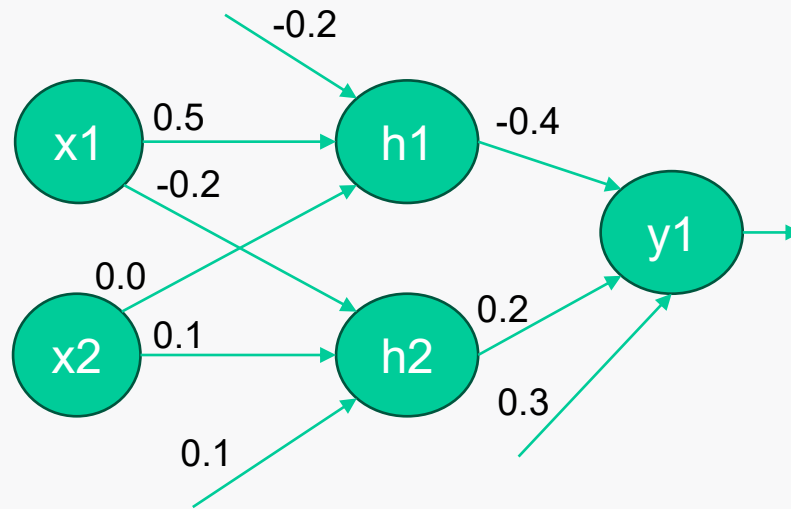
U	
-0.4	0.2

$b^{[2]}$
0.3

Introdução (2/3)

- Nesta aula

- A presente aula cobre a **etapa de treinamento** de uma RNFF para classificação binária.
- Ou seja: veremos como os parâmetros **W** , **U** , **$b^{[1]}$** e **$b^{[2]}$** (conjunto de pesos e termos de viés) são determinados.



W	
0.5	0.0
-0.2	0.1

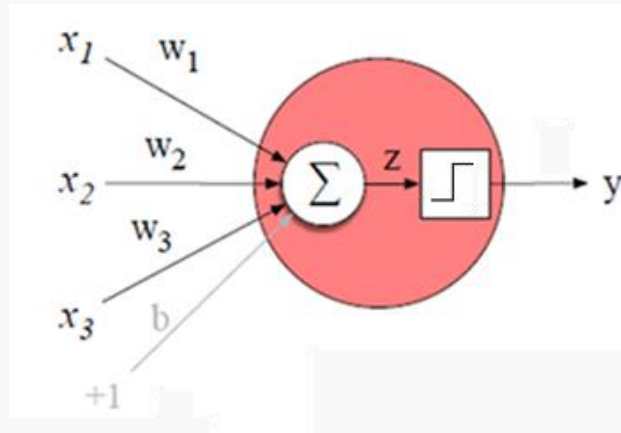
$b^{[1]}$
-0.2
0.1

U	
-0.4	0.2

$b^{[2]}$
0.3

Introdução (3/3)

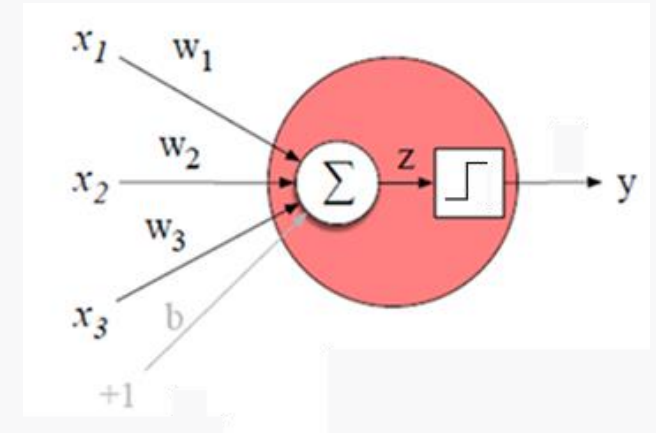
- Mas antes disso...
 - Serão apresentados dois conceitos importantes
 - função de custo
 - gradiente descendente
 - E como ambos são utilizados para treinar um **Perceptron**



Perceptron (1/5)

- **Perceptron – relembrando**

- Conforme mostrado na aula anterior, o Perceptron corresponde uma única unidade neural.
- Ele possui um vetor de pesos $\mathbf{w}$ (no exemplo ao lado $[w_1, w_2, w_3]$) e um termo de viés b .
- Sua tarefa principal é **classificar os dados em duas classes** (classificação binária)

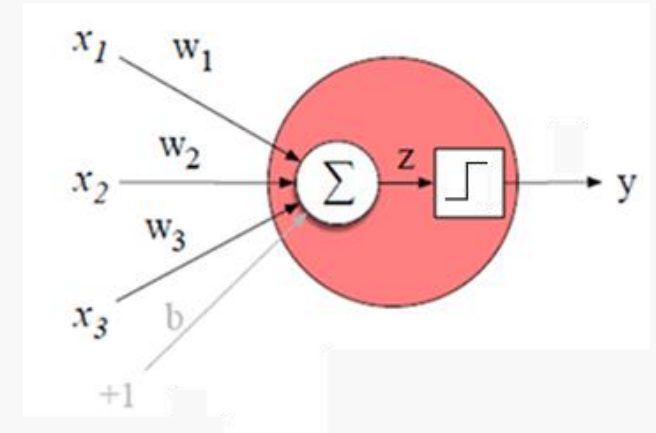


Perceptron (2/5)

- **Perceptron – relembrando**
 - O Perceptron é extremamente simples.
 - Não aplica função de ativação sobre z (a entrada), mas sim a **função degrau**,
 σ .
 - A função degrau possui saída binária.

$$\sigma(z) = \begin{cases} 0, & \text{se } \mathbf{wx} + b \leq 0 \\ 1, & \text{se } \mathbf{wx} + b > 0 \end{cases}$$

- **Obs.:** o Perceptron é mais simples do que as atuais unidades computacionais que formam as redes neurais modernas ([ver aula 12](#)).



Perceptron (3/5)

- Como os parâmetros w (vetor de pesos) e b (viés) são aprendidos no Perceptron?
 - O Perceptron é um classificador binário, logo precisa ser treinado a partir de uma **base rotulada**.
 - BD de treinamento em que cada objeto x está rotulado com sua classe y (0 ou 1)
 - Quando um Perceptron é usado para classificar um objeto, ele computa $\hat{y}$ (**classe estimada** do verdadeiro y).
 - Desta forma, o que deseja-se é:
 - Aprender os parâmetros w e b capazes de fazer com que o $\hat{y}$ de cada objeto de treino seja o mais próximo possível de seu y verdadeiro.

Perceptron (4/5)

- Para tal, o Perceptron usa 2 componentes no treinamento
 1. Uma **função de custo** a ser minimizada
 - Ela avalia o quão distante $\hat{y}$ está do *gold label* y considerando todos os objetos de treino.
 - Adotaremos a minimização da **Soma de Quadrados do Erro (SQE)**
 2. Um **algoritmo** para otimizar a função de perda
 - Realiza, de forma iterativa, a atualização dos pesos com o intuito de minimizar a função de custo.
 - Usaremos o **Gradiente Descendente Estocástico (GD)**
- Existem diversas funções de custo e algoritmos de otimização. A SQE e GD são adotadas em muitos livros, como de Oliveira e Melo Junior (2024).

Perceptron (5/5)

- **Notação**

- Os slides a seguir abordarão o treinamento do Perceptron com a SQE e GD. A seguinte notação é adotada:
 - $D \rightarrow$ base de dados de treinamento de dimensão $m \times n$
 - $m \rightarrow$ número de objetos
 - $n \rightarrow$ número de atributos (*features*)
 - Cada objeto da base é da forma $(\mathbf{x}, y)$, onde $\mathbf{x}$ é o vetor de features e y o rótulo de classe.
 - $(\mathbf{x}^{(k)}, y^{(k)}) \rightarrow$ k -ésimo objeto da base de treinamento
 - $\mathbf{x}^{(k)} \rightarrow$ *features* do k -ésimo objeto
 - $y^{(k)} \rightarrow$ classe do k -ésimo objeto
 - $x_i \rightarrow i$ -ésimo atributo de $\mathbf{x}$

Soma do Quadrado dos Erros – SQE (1/6)

• Componente 1 do treinamento: Função de Custo

- Dado um objeto de treino k , o **erro** $e^{(k)}$ na saída do Perceptron para esse objeto é obtido pela diferença entre o valor real $y^{(k)}$ da classe do objeto e o valor predito $\hat{y}^{(k)}$.
 - $e^{(k)} = y^{(k)} - \hat{y}^{(k)}$
- A **SQE** é uma maneira de quantificar o erro considerando todos os objetos da BD.
 - Dada uma base de treino com m objetos, a SSE é obtida elevando ao quadrado as diferenças entre o valor real e o valor predito (erro) de todos os objetos e, em seguida, somando esses quadrados.

$$SQE = \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2 \quad \text{como } y^{(k)} - \hat{y}^{(k)} = e^{(k)}, \text{ podemos também representar por} \quad \rightarrow \quad SQE = \sum_{i=1}^m (e^{(k)})^2$$

Soma do Quadrado dos Erros – SQE (2/6)

- Quanto menor o valor da SQE, melhor
 - Um valor baixo, indica que o Perceptron está gerando estimativas próximas das classes reais.
 - Se o Perceptron **não errar nada**, a SQE obtida será **0**.
 - Quanto mais distantes os valores preditos forem dos reais, maior será a SQE.
 - Vamos ver alguns exemplos utilizando a BD abaixo (da função AND)

$$SQE = \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2$$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

Soma do Quadrado dos Erros – SQE (3/6)

- **SQE – exemplos**

- Observe que as classes reais (coluna y) são $y^{(1)}=0$, $y^{(2)}=0$, $y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

- **Exemplo 1 – o Perceptron acertou tudo! 😊**

- Suponha que o perceptron previu $\hat{y}^{(1)}=0$, $\hat{y}^{(2)}=0$, $\hat{y}^{(3)}=0$ e $\hat{y}^{(4)}=1$
 - Ou seja, ele acertou tudo!
 - A SQE nesse caso resulta em zero:

$$\begin{aligned}
 SQE &= \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 = \\
 &= (0 - 0)^2 + (0 - 0)^2 + (0 - 0)^2 + (1 - 1)^2 = \mathbf{0.0}
 \end{aligned}$$

Soma do Quadrado dos Erros – SQE (4/6)

- **SQE – exemplos**

- Classes reais (coluna y) $\rightarrow y^{(1)}=0, y^{(2)}=0, y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

- **Exemplo 2 – o Perceptron errou tudo!** 😞

- Suponha agora que o perceptron previu $\hat{y}^{(1)}=1, \hat{y}^{(2)}=1, \hat{y}^{(3)}=1$ e $\hat{y}^{(4)}=0$
 - Ou seja, ele errou tudo!
 - Nesse caso, a SQE terá **valor máximo** para a base de dados

$$\begin{aligned}
 SQE &= \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 = \\
 &= (0 - 1)^2 + (0 - 1)^2 + (0 - 1)^2 + (1 - 0)^2 = \mathbf{4.0}
 \end{aligned}$$

Soma do Quadrado dos Erros – SQE (5/6)

- **SQE – exemplos**

- Classes reais (coluna y) $\rightarrow y^{(1)}=0, y^{(2)}=0, y^{(3)}=0$ e $y^{(4)}=1$

AND		
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

- **Exemplo 3 – o Perceptron acertou metade! ☹️**

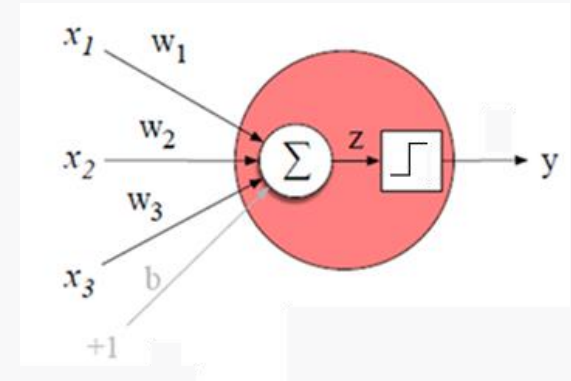
- Dessa vez, considere que o perceptron previu $\hat{y}^{(1)}=1, \hat{y}^{(2)}=1, \hat{y}^{(3)}=0$ e $\hat{y}^{(4)}=1$
 - Errou as 2 primeiras previsões e acertou as 2 últimas.
 - A SQE será computada por:

$$\begin{aligned}
 SQE &= \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2 = (y^{(1)} - \hat{y}^{(1)})^2 + (y^{(2)} - \hat{y}^{(2)})^2 + (y^{(3)} - \hat{y}^{(3)})^2 + (y^{(4)} - \hat{y}^{(4)})^2 = \\
 &= (0 - 1)^2 + (0 - 1)^2 + (0 - 0)^2 + (1 - 1)^2 = \mathbf{2.0}
 \end{aligned}$$

- Como toda função de custo, a SQE tem suas vantagens e desvantagens, conforme discutido em <https://estatisticafacil.org/glossario/o-que-e-sum-of-squared-errors-soma-dos-erros-quadraticos/>

Soma do Quadrado dos Erros – SQE (6/6)

- O erro é função dos pesos
 - Se os pesos do Perceptron estiverem com “bons valores”...
 - ... o valor calculado para z ficará “bom”
 - ... e o erro associado a cada objeto k será baixo
 - ... e, por consequência, a SQE ficará baixa.
 - Mas se os pesos forem “ruins”, o erro vai aumentar (o Perceptron vai errar mais classificações)...
 - ... com o consequente aumento da SQE.
 - Precisamos então de um processo para alcançar o valor desejado para os pesos.
 - É aí que entrará o Gradiente Descendente



$$z = \left(\sum_{i=1}^n w_i x_i \right) + b$$

$$SQE = \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2$$

Gradiente Descendente – Intuição (1/5)

- O que é gradiente descendente (GD) ?
 - É um processo iterativo para alcançar o valor desejado para os pesos baseado no gradiente do erro (*apresentaremos em breve a noção de gradiente*).
 - **Melhor dizendo:** o objetivo do GD é determinar os **parâmetros** que minimizam o erro do modelo (Perceptron, RNFF ou outra RN) considerando todos os objetos de treino.
 - Os parâmetros são denotados por θ (convenção nas áreas de IC e ML).
 - No caso do Perceptron, $\theta = \mathbf{w}, b$

Gradiente Descendente – Intuição (2/5)

- Como o GD funciona ?

- Em um dado ponto de referência (ponto corrente), o GD descobre em que direção, no espaço de parâmetros θ , a **inclinação** da função de custo (denotada por L) cresce de forma **mais acentuada**.
- Depois de descobrir, ele **se move**, a partir do ponto corrente, **na direção oposta**.
 - Pois o objetivo é chegar ao mínimo de custo e não ao máximo !
- Nos slides a seguir, representaremos o valor da função de custo para um objeto específico $\mathbf{x}^{(k)}$ como $L(f(\mathbf{x}^{(k)}; \theta), y^{(k)})$
 - Acima, $\hat{y}^{(k)}$ é representado como $\mathbf{f}(\mathbf{x}^{(k)}, \theta)$
 - A notação explícita o fato de que a função de perda L é **parametrizada** por θ .

Gradiente Descendente – Intuição (3/5)

- GD – Intuição

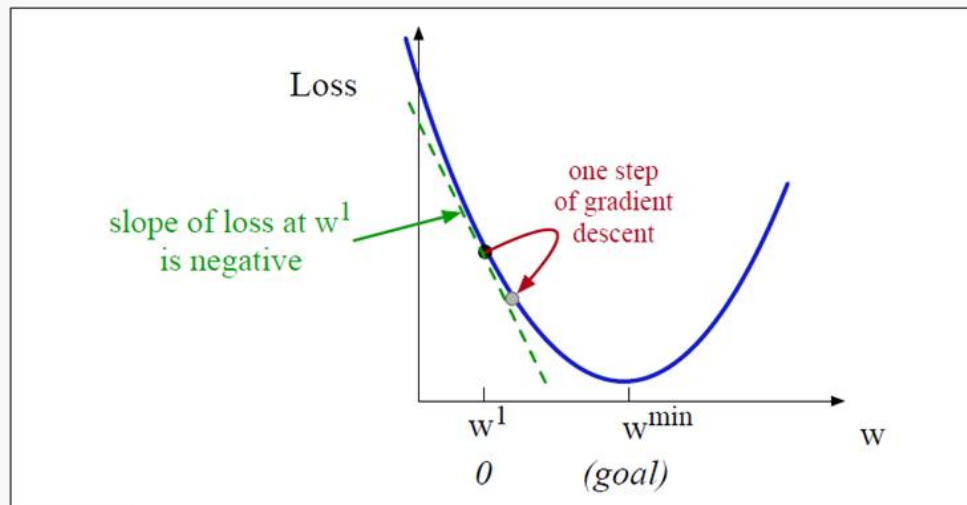
- Imagine que você é um alpinista que quer descer uma montanha o mais rápido possível.
- Você olha 360° em volta e identifica onde o terreno é mais inclinado
- E desce alguns passos nessa direção (*no mundo das RNs, o passo é chamado de taxa de aprendizagem e denotado por η*)
- E repete o processo, até conseguir completar a descida.



Gradiente Descendente – Intuição (4/5)

- **GD – Exemplo**

- O exemplo abaixo mostra um passo do GD em um problema de um único parâmetro w .
 - **Dados:** um valor inicial aleatório w^1 para w ; e a função de custo L com a forma abaixo.
 - O GD nos diz **como** devemos **nos mover** para **alcançar o mínimo**:
 - para a **esquerda** (fazendo w^2 ser menor do que w^1) ?
 - **ou** para a **direita** (fazendo w^2 ser maior do que w^1) ?

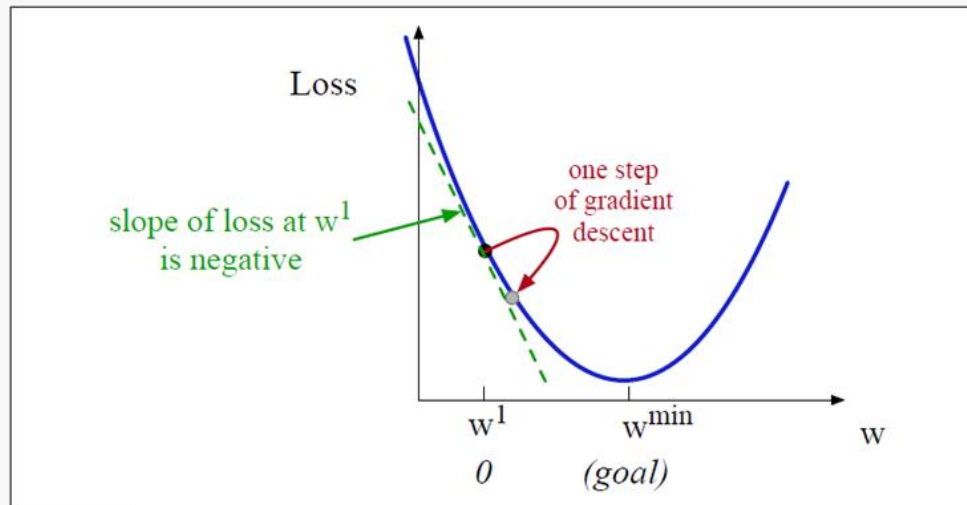


Fonte:
Jurafsky e Martin (2025)

Gradiente Descendente – Intuição (5/5)

• GD – Exemplo

- A reta pontilhada indica a taxa de variação de L (Loss, eixo Y) em relação a w (eixo X) no instante w^1 .
- Como a inclinação da reta no ponto w^1 é negativa nos **moveríamos** para a **direita** (w^2 deve ser maior do que w^1).



Gradiente Descendente (1/8)

- Mas como e o quanto devemos nos mover ?
 - O GD responde a essa questão encontrando o **gradiente** da função de perda no ponto corrente e fazendo o movimento na direção contrária.
 - Gradiente de uma função multivariada (com muitas variáveis) → vetor apontando na direção da inclinação mais acentuada da função.
 - A quantidade a ser movida é controlada pela **taxa de aprendizagem η** .
 - η é um **hiperparâmetro** que é especificado pelo usuário.
 - Um valor alto significa que o movimento de **w** será mais largo a cada passo (vai “andar mais rápido”, porém com risco de passar do mínimo)

Gradiente Descendente (2/8)

- Mas o quanto devemos nos mover ?

- Para uma função com muitas variáveis (multivariada), a mudança no valor dos parâmetros θ , é determinada por: $\theta^{t+1} = \theta^t - \eta \nabla L(f(\mathbf{x}; \theta), y)$
 - $\theta^{t+1} \rightarrow$ próximo valor de θ (valor no instante $t+1$)
 - $\theta^t \rightarrow$ valor atual de θ
 - $\eta \rightarrow$ taxa de aprendizagem (controla o tamanho do “passo” que será dado, normalmente valor no intervalo $0 < \eta \leq 1$)
 - $\nabla L(f(\mathbf{x}; \theta), y) \rightarrow$ gradiente da função de custo no ponto corrente.
- **Ou seja:** a partir de onde estou agora (θ^t), devo fazer η movimentos na direção oposta a que o gradiente indica ($-\nabla L(f(\mathbf{x}; \theta), y)$) para chegar ao novo local (θ^{t+1})

Gradiente Descendente (3/8)

- **Gradiente de uma Função**

- No exemplo dos slides 20 e 21, tínhamos somente uma variável.
 - Foi então possível pensar informalmente no gradiente como a inclinação de uma reta.
 - Dessa forma, para achar o mínimo, podemos nos mover para esquerda ou direita.
- Porém, em problemas reais, o número de variáveis é muito maior !!!
 - Por isso, na verdade, precisamos saber **onde devemos nos mover, no espaço N -dimensional** (dos N parâmetros que compõem θ).
 - Sendo assim, o gradiente será um **vetor** ...
 - ... que expressa os componentes direcionais da inclinação mais acentuada entre cada uma das N dimensões.

Gradiente Descendente (4/8)

- **Gradiente de uma Função**

- Para cada dimensão w_i em $\mathbf{w}$ (mais o intercepto b), o vetor gradiente terá um componente que nos diz a inclinação com respeito a essa dimensão.
- Em cada dimensão w_i , expressamos a **inclinação** como a **derivada parcial** $\frac{\partial}{\partial w_i}$ da função de custo.
 - “*O quanto uma pequena mudança em w_i influencia na função de custo L como um todo?*”

Gradiente Descendente (5/8)

- **Gradiente de uma Função**

- Usa-se o símbolo ∇ para referir ao gradiente.
- O gradiente de uma função multivariada f é um **vetor** em que cada componente expressa a derivada parcial de f com respeito a uma das variáveis.

- $$\nabla L(f(x; \theta), y) = \begin{bmatrix} \frac{\partial}{\partial w_1} L(f(x; \theta), y) \\ \vdots \\ \frac{\partial}{\partial w_n} L(f(x; \theta), y) \\ \frac{\partial}{\partial b} L(f(x; \theta), y) \end{bmatrix}$$

- Relembrando que na fórmula acima, $\hat{y}$ foi representado como $f(x; \theta)$ para deixar clara sua dependência em relação a θ .
- Conforme apresentado, a **equação final** para **atualizar** θ no passo $t+1$ é:

- $$\theta^{t+1} = \theta^t - \eta(\nabla L(f(x; \theta), y))$$

Gradiente Descendente (6/8)

- Mas como fica a derivada da função de perda?

- Em nosso exemplo, a função de perda é a SQE: $L(y, \hat{y}) = \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2$
- Por conveniência (*para combinar melhor com a derivação que será feita*), vamos dar uma pequena mexida e adotar **metade da soma dos quadrados dos erros** como função de perda:

$$L(y, \hat{y}) = \frac{1}{2} \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2$$

Gradiente Descendente (7/8)

- Mas como fica a derivada da função de perda?

$$L(y, \hat{y}) = \frac{1}{2} \sum_{i=1}^m (y^{(k)} - \hat{y}^{(k)})^2$$

- A derivada da função L acima para **uma observação x** em relação à variável j é:
 - $\frac{\partial L(\hat{y}, y)}{\partial w_j} = \frac{1}{2} \times 2(\hat{y} - y)x_j = (\hat{y} - y)x_j$ ou $-(y - \hat{y})x_j$
- Intuitivo!
 - O gradiente com respeito a um único peso w_j é a **diferença entre o y real e o $\hat{y}$ estimado** para a observação, **multiplicado pelo valor de entrada correspondente x_j** .

Gradiente Descendente (8/8)

- **Equação final**

- Agora que sabemos calcular o gradiente da função de custo, a **equação final** para **atualizar** θ no passo $t+1$ é:

- $\theta^{(t+1)} = \theta^{(t)} - \eta(\nabla L(f(x; \theta), y)),$

- $\theta^{(t+1)} = \theta^{(t)} - \eta(-[y^{(t)} - \hat{y}^{(t)}] x^{(t)}),$

- $\theta^{(t+1)} = \theta^{(t)} + \eta(e^{(t)} x^{(t)})$

Treinamento do Perceptron – um exemplo (1/5)

• Exemplo – Classificador de Sentimentos

- Vamos apresentar um exemplo de uma etapa de treinamento, que corresponde ao modelo recebendo a primeira observação da base de dados ao lado.
- É uma **BD de treino hipotética** p/ classificação de sentimentos com duas variáveis:
 - x_1 → contagem de palavras positivas
 - x_2 → contagem de palavras negativas.
- Com relação ao atributo classe y :
 - **0** → sentimento negativo
 - **1** → sentimento positivo

x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1

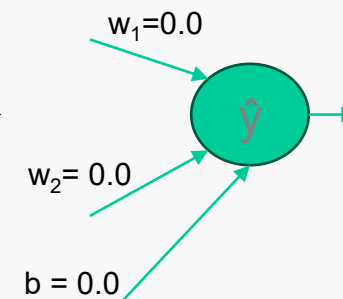
Treinamento do Perceptron – um exemplo (2/5)

• Exemplo – Classificador de Sentimentos

- Vamos apresentar um exemplo de uma etapa de treinamento.
- Será o modelo recebendo **a primeira observação** da base → **[3, 2]**
 - $x_1 = 3$ (contagem de palavras positivas)
 - $x_2 = 2$ (contagem de palavras negativas).
 - A classe real da observação é $y = 1$.

x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1

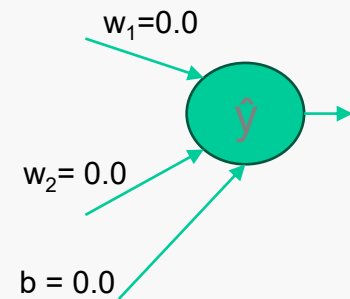
- O exemplo vai mostrar o **primeiro passo do treinamento** do Perceptron. Considere a seguinte configuração:
 - $\theta^{(0)} = w_1 = w_2 = b = 0$ (pesos iniciais)
 - $\eta = 0.1$ (taxa de aprendizado)



Treinamento do Perceptron – um exemplo (3/5)

- **Exemplo – Classificador de Sentimentos**

- Etapa de atualização p/ observação: $\mathbf{x} = [3, 2]$, $y = 1$.
- Sendo: $\theta^{(0)} = w_1 = w_2 = b = 0.0$ $\eta = 0.1$
- PASSO 1: considerando os parâmetros atuais $\theta^{(0)}$, e o vetor $\mathbf{x}$ atual, calculamos a saída $\hat{y}$ gerada pelo perceptron:
 - $z = w_1 x_1 + w_2 x_2 + b = 0.0 \times 3 + 0.0 \times 2 + 0.0 = 0.0$
 - $\hat{y} = \text{sig}(z) = 0.0$
- PASSO 2: calculamos o erro na saída:
 - $e = y - \hat{y} = 1 - 0 = 1.0$
 - *Podemos ver que, para os parâmetros atuais, o Perceptron produziu uma saída diferente do valor real da classe.*



Treinamento do Perceptron – um exemplo (4/5)

• Exemplo – Classificador de Sentimentos

- Etapa de atualização p/ observação:

$$x_1 = 3, x_2 = 2, y = 1$$

- Sendo: $\theta^{(0)} = w_1 = w_2 = b = 0$ $\eta = 0.1$

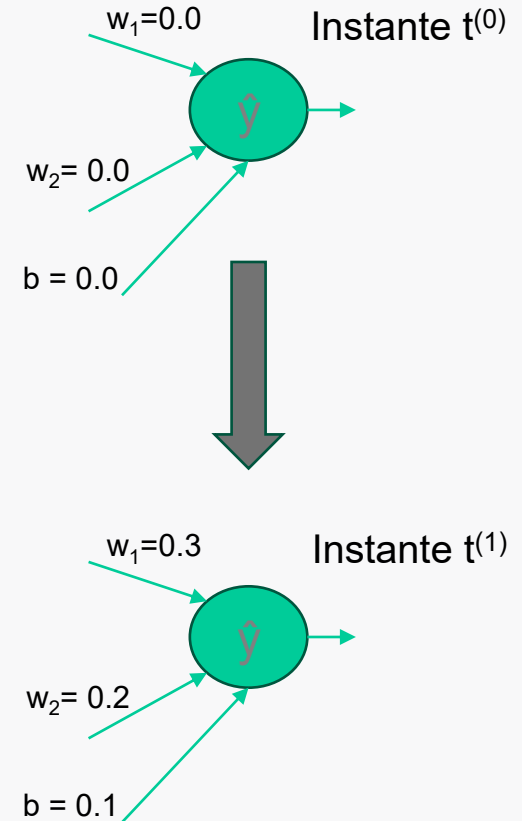
- PASSO 3: cálculo dos novos valores de w_1 , w_2 e b

- $w_1^{(1)} = w_1^{(0)} + \eta(e^{(0)} x_1^{(0)}) = 0 + 0.1 \times (1.0 \times 3) = 0.3$
- $w_2^{(1)} = w_2^{(0)} + \eta(e^{(0)} x_2^{(0)}) = 0 + 0.1 \times (1.0 \times 2) = 0.2$
- $b^{(1)} = b^{(0)} + \eta(e^{(0)}) = 0 + 0.1 (1) = 0.1$

- Novos parâmetros $\theta^{(1)}$:

- $w_1 = 0.3$
- $w_2 = 0.2$
- $b = 0.1$

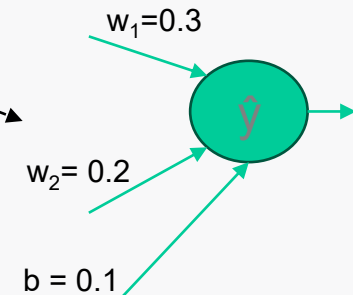
- Ou seja: depois de 1 passo do GD, os parâmetros mudaram p/ $w_1 = 0.3$, $w_2 = 0.2$ e $b = 0.1$



Treinamento do Perceptron – um exemplo (5/5)

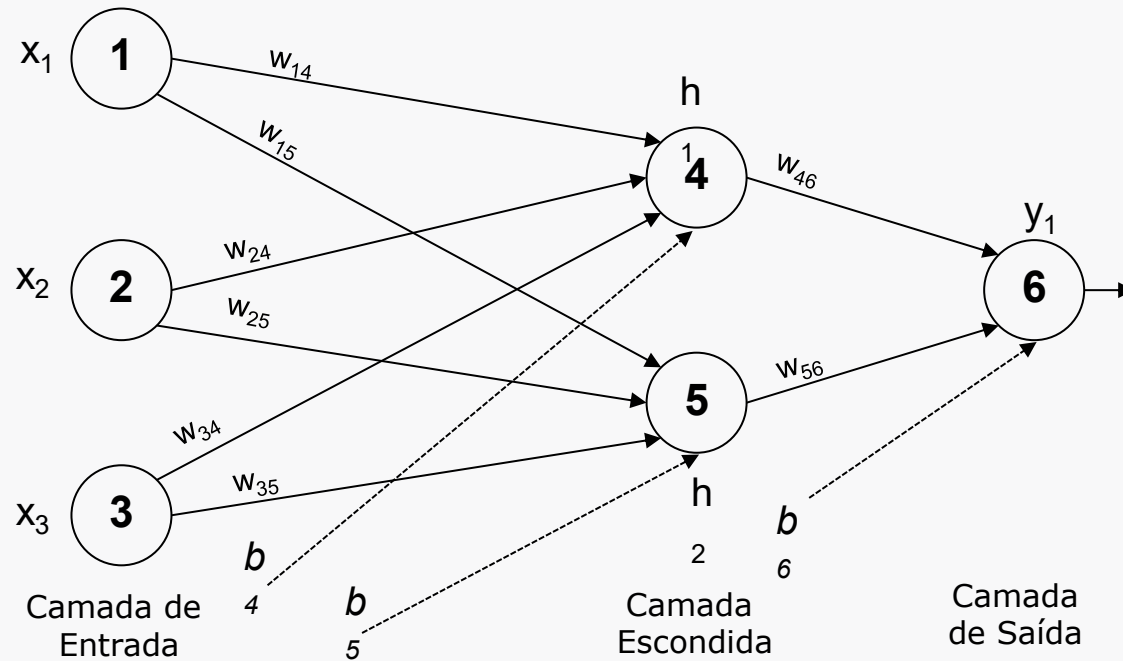
- Como o algoritmo vai continuar ?
- O algoritmo prossegue pegando a próxima observação
- E realizando o mesmo processamento, mas agora com os novos pesos
- Quando processar a última observação da BD de treino, dizemos que acabou uma **época**.
- Então, ele vai para a 2ª época (processa toda a base de dados de novo). E fará isso:
 - por um número fixo de épocas (ex.: 1000 épocas)
 - ou até o erro de uma época ficar muito baixo (abaixo de um limiar)
 - Ou até os pesos estarem mudando muito pouco (abaixo de um limiar)

x_1 (contagem de palavras positivas)	x_2 (contagem de palavras negativas)	y (classe)
3	2	1
5	0	1
0	1	0
7	2	1
1	3	0
...	...	...
3	0	1



RNFF (1/4)

- Mas como treinar uma RNFF – rede neural feed forward ?
 - Acabamos de ver o treinamento do Perceptron.
 - Mas como a RNFF é treinada ?



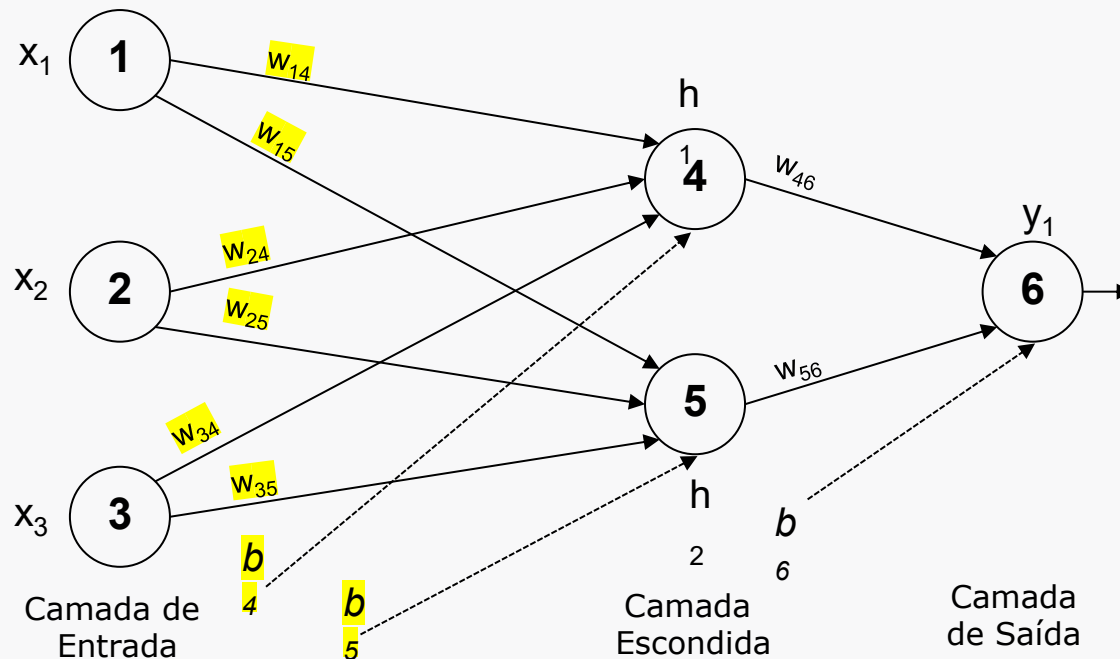
RNFF (2/4)

- Resposta: com o algoritmo da **Retropropagação** (*backpropagation*)
 - Também é baseado em função de perda e gradiente descendente
 - O nome retropropagação vem do fato de que o erro na camada de saída é “propagado para trás” na rede para permitir o ajuste de todos os pesos.
 - **É bem parecido** com o treinamento do Perceptron...
 - ... no processo de treinamento, a RNFF aprende **ajustando sucessivamente os pesos** de cada camada p/ tornar o **$\hat{y}$ predito** (saída da rede) para cada objeto de treino o mais próximo possível do **y real**.

RNFF (3/4)

- **Retropropagação** (*backpropagation*)

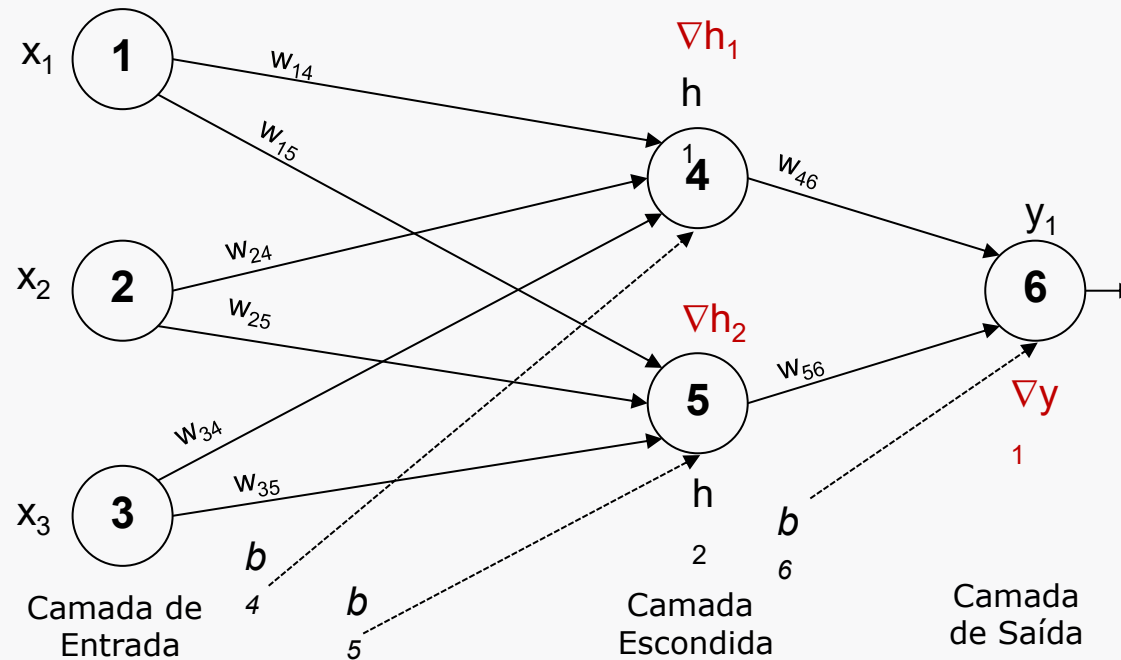
- **Porém...** a retropropagação tem fundamentação teórica mais sofisticada, utilizando conceito de derivadas parciais em grafos de computação (Jurafsky e Martin, 2025).
- Isso porque é preciso atualizar os pesos da camada escondida. No Perceptron só temos uma equação a otimizar e na RNFF temos várias.



RNFF (5/5)

- Agora são vários gradientes

- No caso da RNFF abaixo, precisaremos calcular 3 gradientes para poder ajustar todos os pesos: ∇h_1 , ∇h_2 e ∇y_1
- Se tivéssemos mais nós na camada escondida ou na de saída, seriam mais gradientes



Retropropagação – um exemplo (1/14)

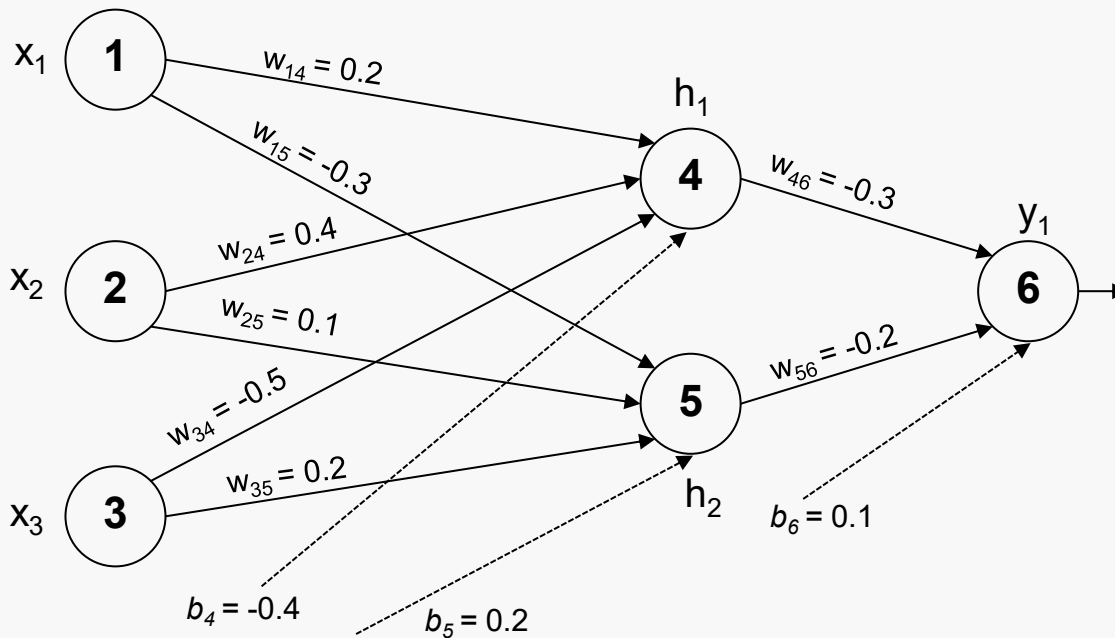
- **Exemplo:** adaptado de Han et al. (2011) e de Oliveira e Melo Junior (2024).
 - Uma iteração de treinamento, correspondendo a uma RNFF recebendo um objeto de treino $\mathbf{x}^{(k)} = [x_1, x_2, x_3]$
 - Considere que:
 - A topologia da rede é a mesma do slide anterior.
 - $x_1 = 1, x_2 = 0, x_3 = 1$ ou: $\mathbf{x} = [1, 0, 1]$
 - A classe real da observação é $y = 1$.
 - A função sigmoide será usada como função de ativação
 - Suponha ainda que:
 - $\eta_0 = 0.9$ (taxa de aprendizado, conforme visto na aula sobre RL)
 - pesos atuais =

w_{14}	w_{15}	w_{24}	w_{25}	w_{34}	w_{35}	w_{46}	w_{56}	b_4	b_5	b_6
0.2	-0.3	0.4	0.1	-0.5	0.2	-0.3	-0.2	-0.4	0.2	0.1

Retropropagação – um exemplo (2/14)

- RN e seus pesos

- Estou mostrando as matrizes W , $b^{[1]}$, U e $b^{[2]}$ também para lembrarmos... No entanto, para o exemplo ficar simples, não farei os cálculos as utilizando.



W			$b^{[1]}$	
0.2	0.4	-0.5	-0.4	
-0.3	0.1	0.2	0.2	

U		$b^{[2]}$	
-0.3	-0.2	0.1	

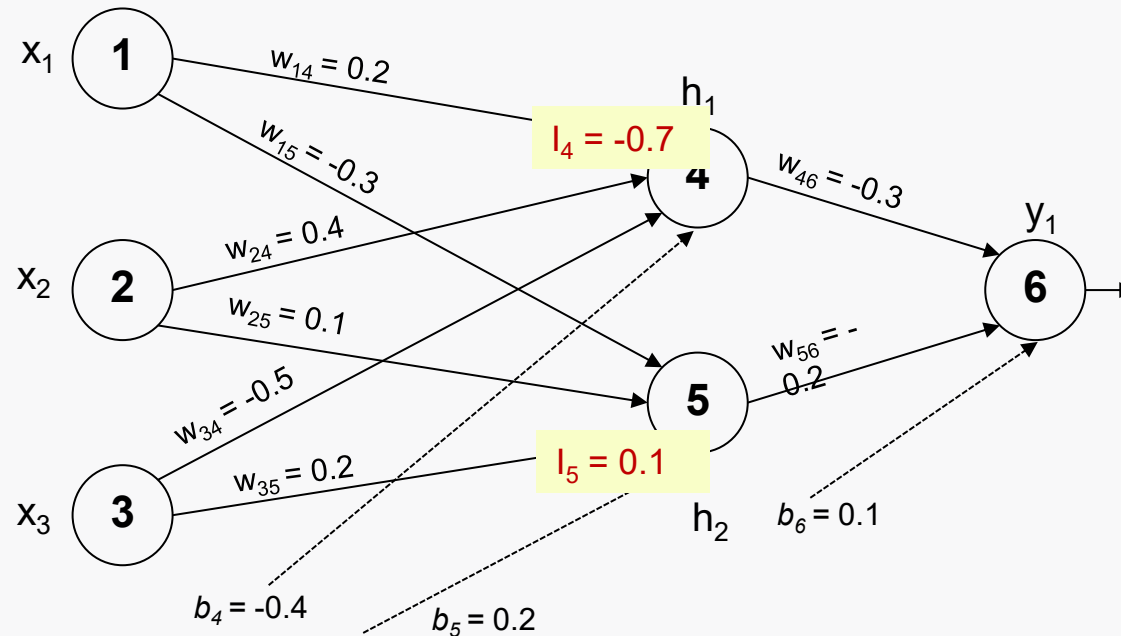
Retropropagação – um exemplo (3/14)

- Passo 1(a): cálculo das entradas das unidades escondidas**

- O objeto de treino $\mathbf{x} = [1, 0, 1]$ é inserido na rede e calculamos l_4 e l_5 .

- $l_4 = x_1 \cdot w_{14} + x_2 \cdot w_{24} + x_3 \cdot w_{34} + b_4 = 1 \times 0.2 + 0 \times 0.4 + 1 \times -0.5 + -0.4 = -0.7$

- $l_5 = x_1 \cdot w_{15} + x_2 \cdot w_{25} + x_3 \cdot w_{35} + b_5 = 1 \times -0.3 + 0 \times 0.1 + 1 \times 0.2 + 0.2 = 0.1$



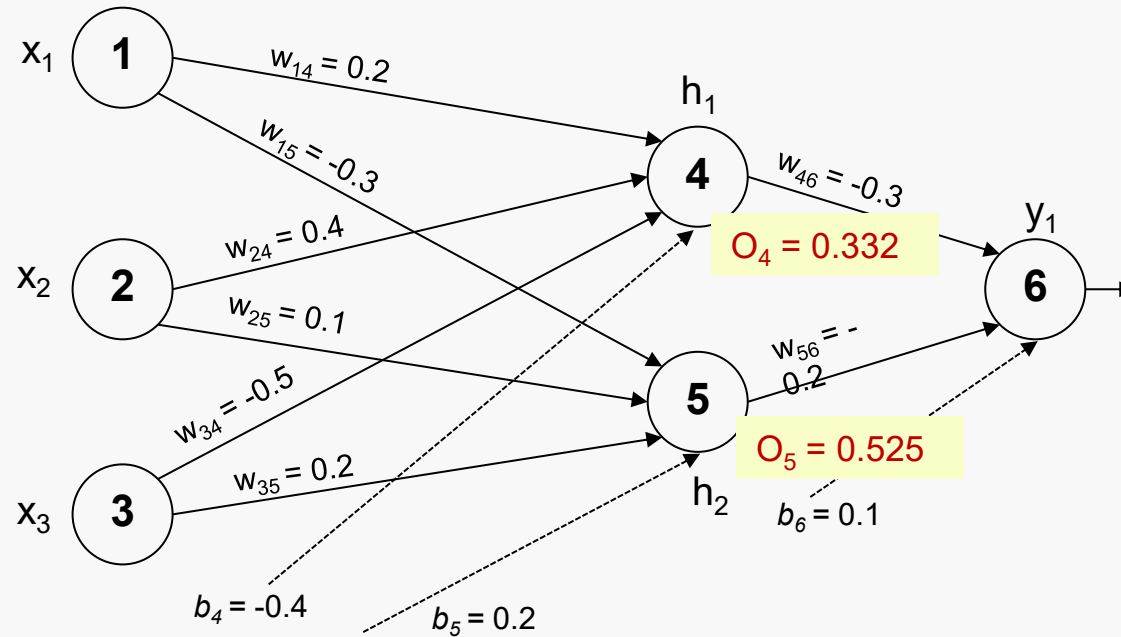
Retropropagação – um exemplo (4/14)

• Passo 1(b): cálculo da saída das unidades escondidas

- Aplicamos a sigmoide sobre l_4 e l_5 , produzindo O_4 e O_5

- $O_4 = \sigma(l_4) = 1 \div (1 + e^{-l_4}) = 1 \div (1 + e^{0.7}) = 0.332$

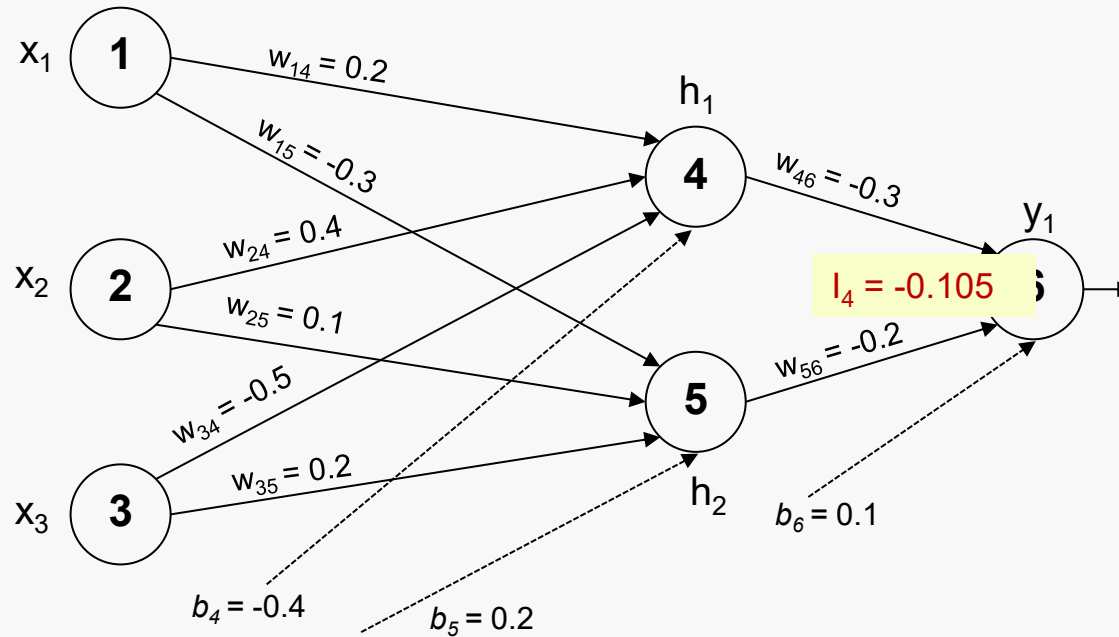
- $O_5 = \sigma(l_5) = 1 \div (1 + e^{-l_5}) = 1 \div (1 + e^{-0.1}) = 0.525$



Retropropagação – um exemplo (5/14)

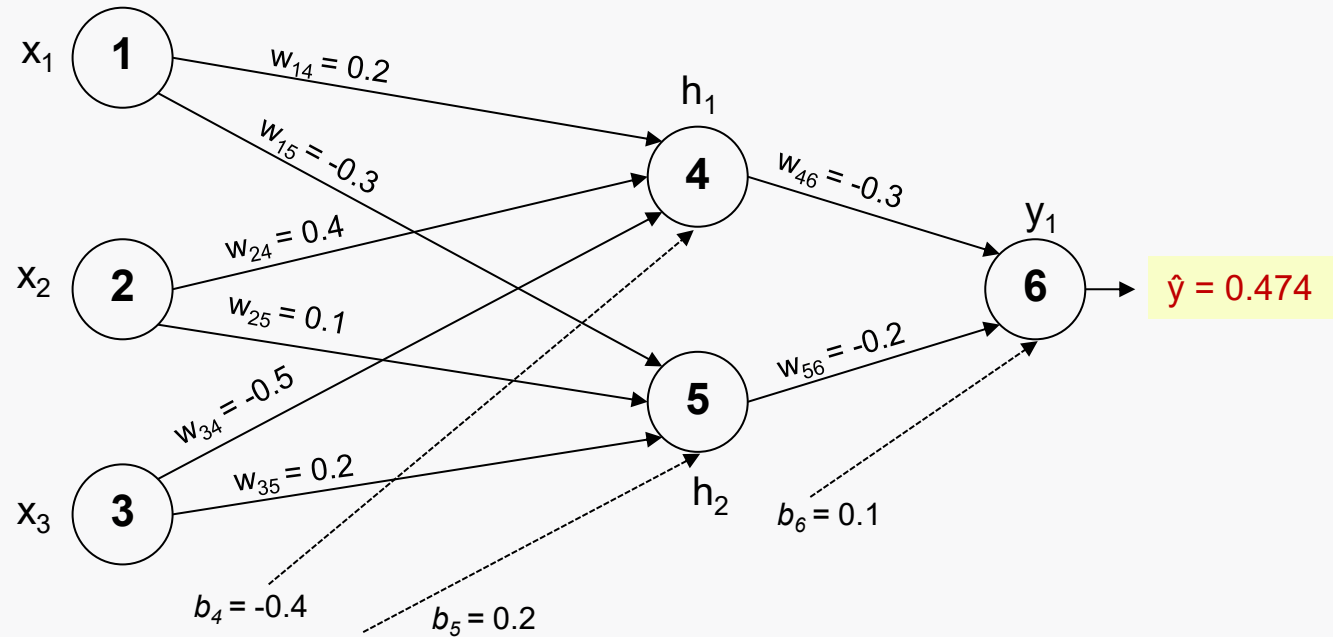
- Passo 2(a): cálculo da entrada da unidade de saída

- $I_6 = O_4 \cdot w_{46} + O_5 \cdot w_{56} + b_6 = 0.332 \times -0.3 + 0.525 \times -0.2 + 0.1 = -0.105$



Retropropagação – um exemplo (6/14)

- Passo 2(b): cálculo da saída $\hat{y}$ gerada pela RN
 - Aplicamos a sigmoide sobre l_6 para obter o $\hat{y}$ predito pela rede
 - $\hat{y} = \sigma(l_6) = 1 \div (1 + e^{-l_6}) = 1 \div (1 + e^{0.105}) = 0.474$



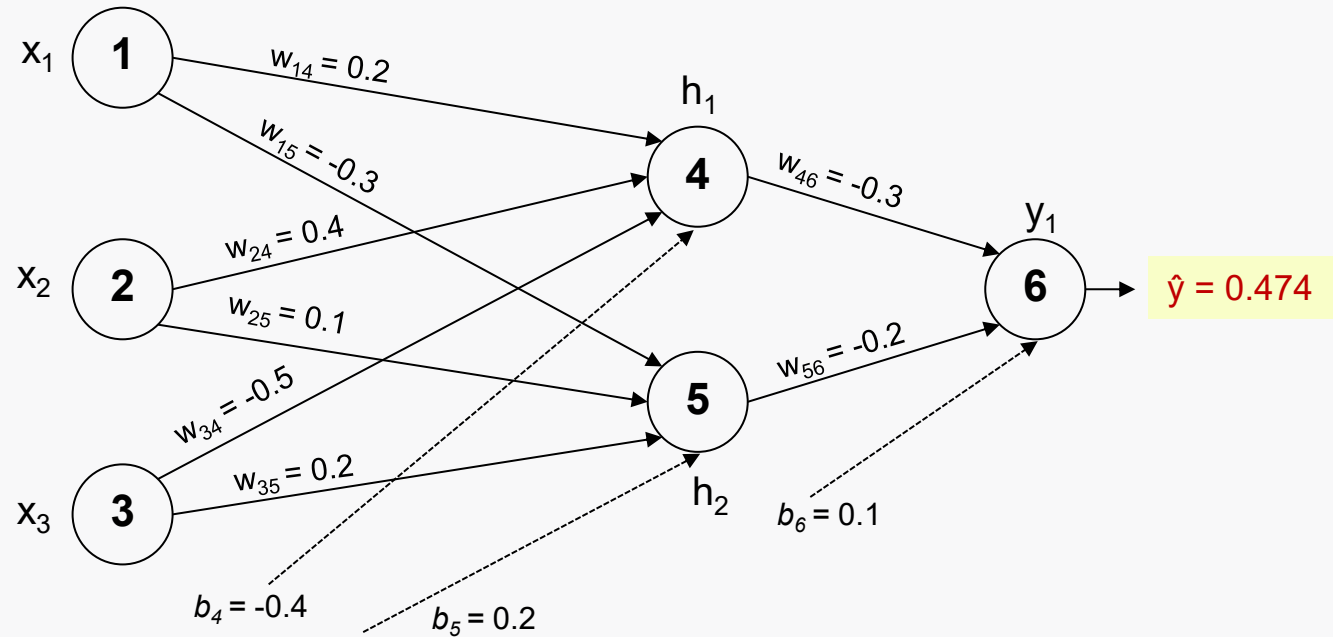
Retropropagação – um exemplo (7/14)

• Passo 3: cálculo do erro na saída

- A classe real da observação é $y = 1$ (ver slide 39)

- $e = y - \hat{y} = 1 - 0.474 = 0.526$

- *Agora vamos começar a retropropagar o erro para ajustar os parâmetros.*

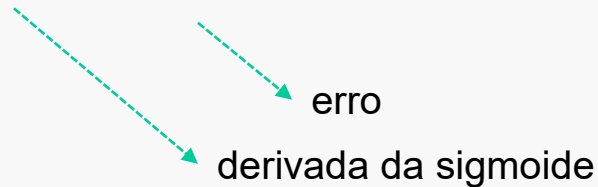


Retropropagação – um exemplo (8/14)

- **Passo 4(a): cálculo do gradiente no neurônio da camada de saída**

- A classe real da observação é $y = 1$ (ver slide 39)

- $\nabla_{y1} = \sigma'(l_6) \times (y - \hat{y})$



- $\nabla_{y1} = [\sigma(l_6) \times (1 - \sigma(l_6))] \times (y - \hat{y}) = [\sigma(-0.105) \times (1 - \sigma(-0.105))] \times (1 - 0.474) =$

- $\nabla_{y1} = [0.474 \times (1 - 0.474)] \times (1 - 0.474) = 0.1311$

- Lembrando que o exemplo está considerando:

- a sigmoide como função de ativação (na unidade de saída e nas escondidas)
- a SQE como função de custo

Retropropagação – um exemplo (9/14)

- **Passo 4(b): cálculo do gradiente nos neurônios da camada escondida**

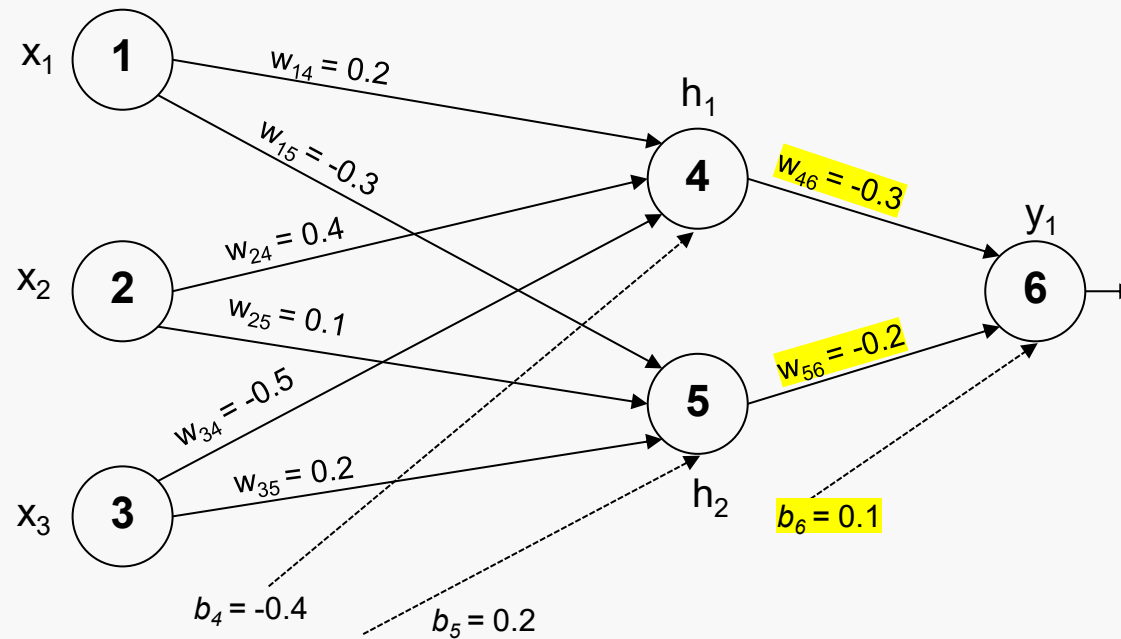
- $\nabla_{h1} = \sigma'(I_4) \times (\nabla_{y1} \times w_{46})$
 - derivada da sigmoide
 - gradiente da unidade de saída
 - peso da conexão da unidade h1 para a unidade y1

- $\nabla_{h1} = [\sigma(I_4) \times (1 - \sigma(I_4))] \times (\nabla_{y1} \times w_{46}) = [\sigma(-0.7) \times (1 - \sigma(-0.7))] \times (0.1311 \times -0.3) =$
- $\nabla_{h1} = [0.332 \times (1 - 0.332)] \times (0.1311 \times -0.3) = -0.0087$

- De maneira análoga, calcula-se ∇_{h2}
 - $\nabla_{h2} = [\sigma(I_5) \times (1 - \sigma(I_5))] \times (\nabla_{y1} \times w_{56}) = [\sigma(0.1) \times (1 - \sigma(0.1))] \times (0.1311 \times -0.2) =$
 - $\nabla_{h2} = [0.525 \times (1 - 0.525)] \times (0.1311 \times -0.2) = -0.0065$

Retropropagação – um exemplo (10/14)

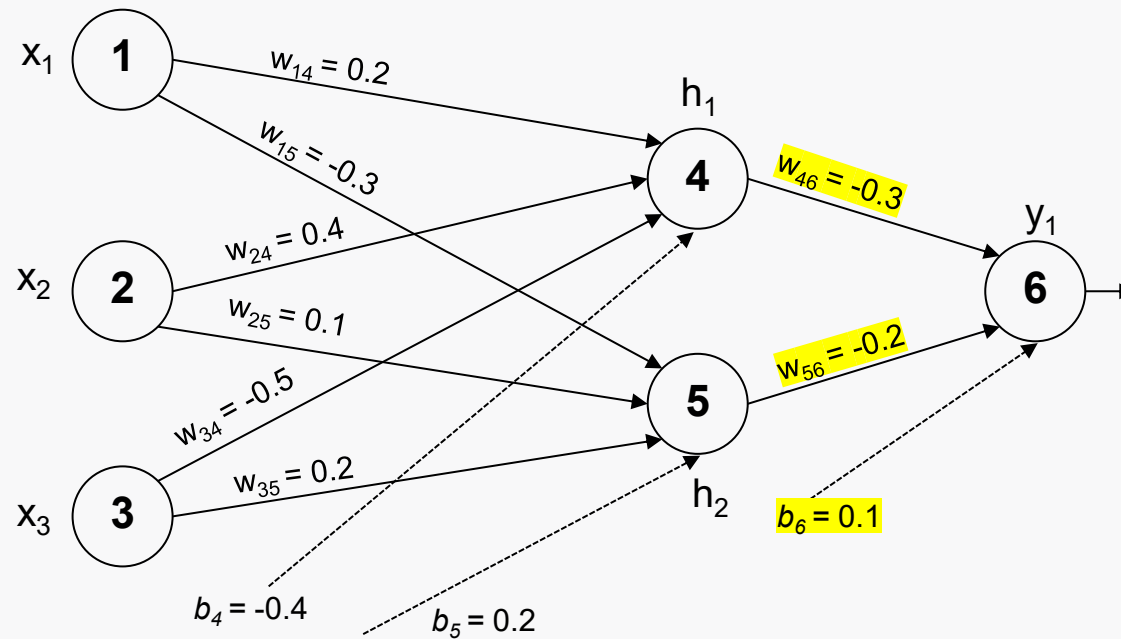
- Passo 5: atualização dos parâmetros (pesos e termos de viés)
 - Feito de maneira bastante similar ao que é feito no treinamento do Perceptron
 - Vamos começar pelos parâmetros w_{46} , w_{56} e b_6 da camada de saída.



Retropropagação – um exemplo (11/14)

• Passo 5: atualização dos parâmetros

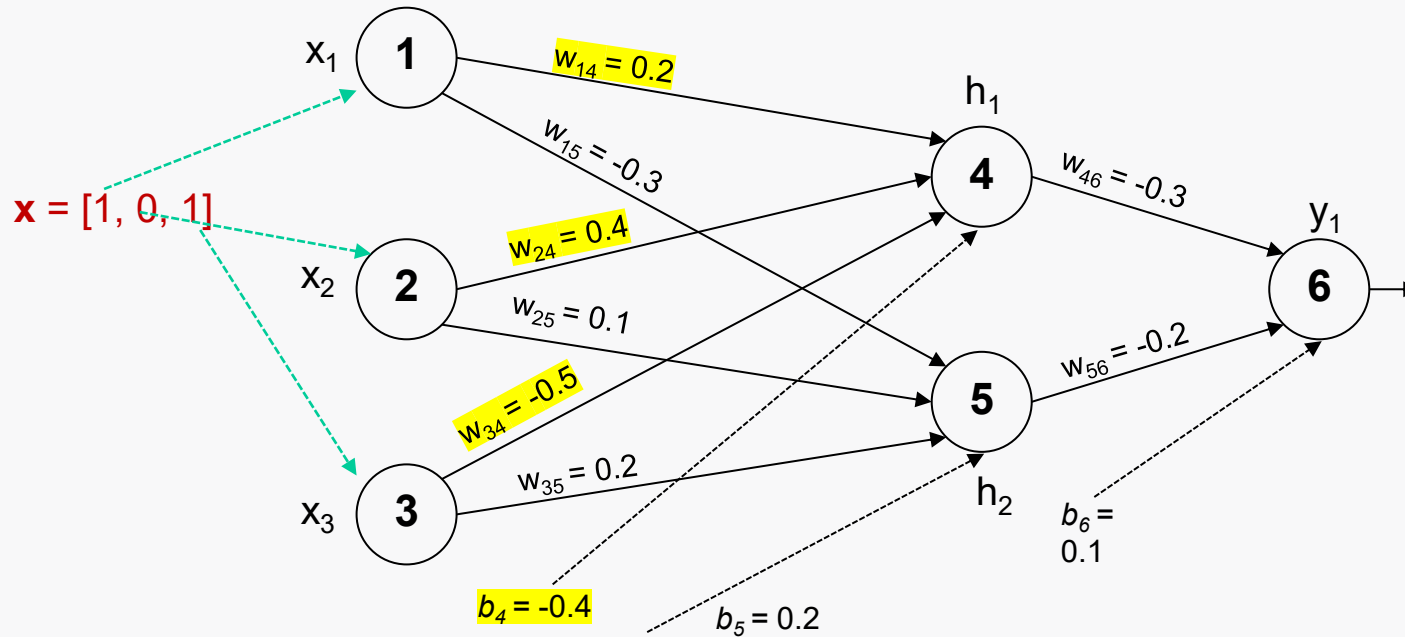
- $w_{46}^{(1)} = w_{46}^{(0)} + \eta(\nabla_{y1} \times O_4) = -0.3 + 0.9 \times (0.1311 \times 0.332) = -0.261$
- $w_{56}^{(1)} = w_{56}^{(0)} + \eta(\nabla_{y1} \times O_5) = -0.2 + 0.9 \times (0.1311 \times 0.525) = -0.138$
- $b_6^{(1)} = b_6^{(0)} + \eta(\nabla_{y1}) = 0.1 + 0.9 \times (0.1311) = 0.218$



Retropropagação – um exemplo (12/14)

• Passo 5: atualização dos parâmetros (cont...)

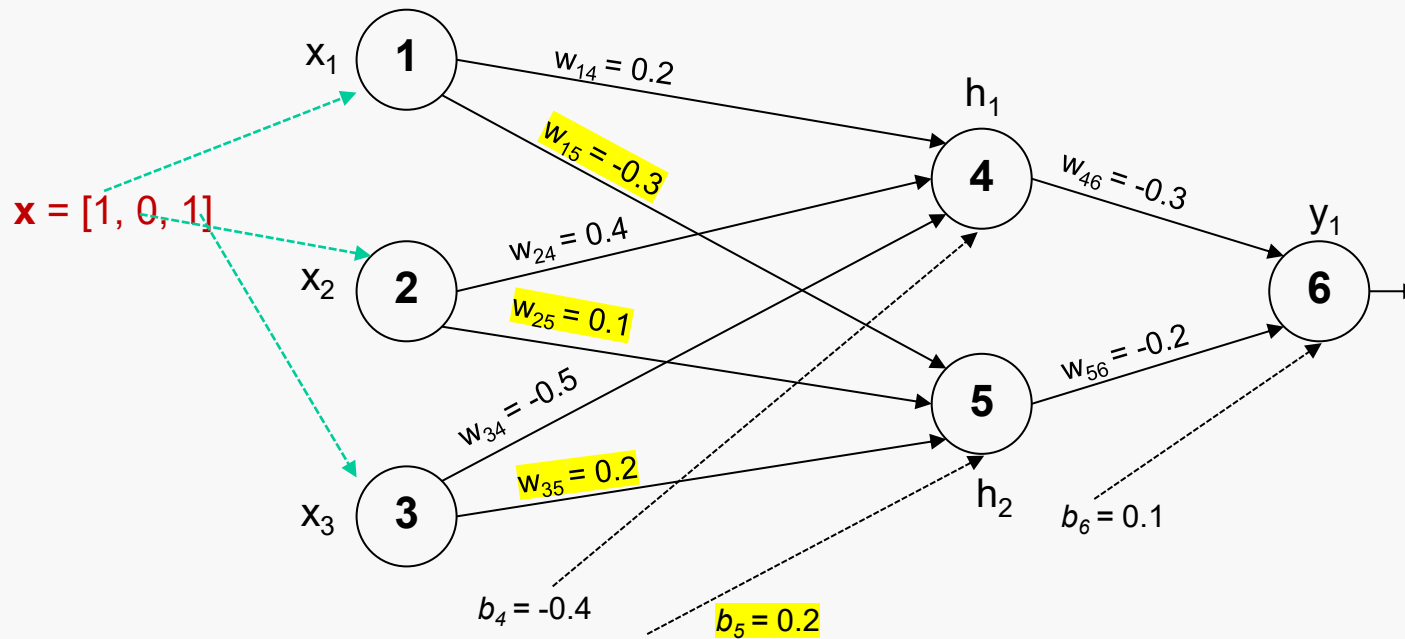
- $w_{14}^{(1)} = w_{14}^{(0)} + \eta(\nabla_{h1} \times x_1) = 0.2 + 0.9 \times (-0.0087 \times 1) = 0.192$
- $w_{24}^{(1)} = w_{24}^{(0)} + \eta(\nabla_{h1} \times x_2) = 0.4 + 0.9 \times (-0.0087 \times 0) = 0.4$
- $w_{34}^{(1)} = w_{34}^{(0)} + \eta(\nabla_{h1} \times x_3) = -0.5 + 0.9 \times (-0.0087 \times 1) = 0.508$
- $b_4^{(1)} = b_4^{(0)} + \eta(\nabla_{h1}) = -0.4 + 0.9 \times (0.-0.0087) = -0.408$



Retropropagação – um exemplo (13/14)

• Passo 5: atualização dos parâmetros (cont...)

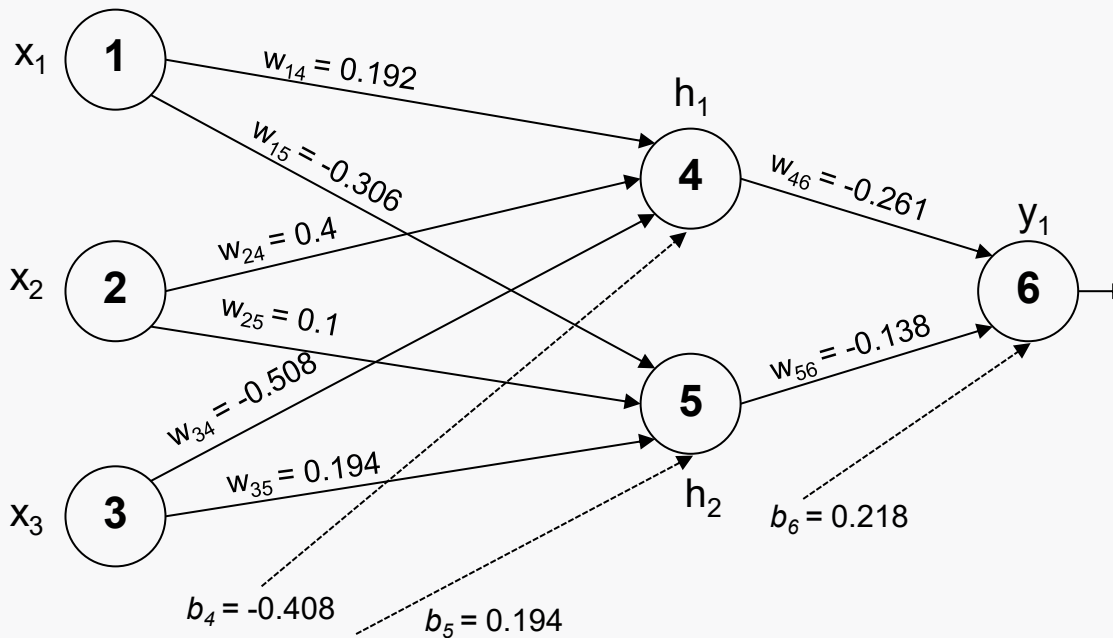
- $w_{15}^{(1)} = w_{15}^{(0)} + \eta(\nabla_{h_2} \times x_1) = -0.3 + 0.9 \times (-0.0065 \times 1) = -0.306$
- $w_{25}^{(1)} = w_{25}^{(0)} + \eta(\nabla_{h_2} \times x_2) = 0.1 + 0.9 \times (-0.0065 \times 0) = 0.1$
- $w_{35}^{(1)} = w_{35}^{(0)} + \eta(\nabla_{h_2} \times x_3) = 0.2 + 0.9 \times (-0.0065 \times 1) = 0.194$
- $b_5^{(1)} = b_5^{(0)} + \eta(\nabla_{h_2}) = 0.2 + 0.9 \times (0. - 0.0065) = 0.194$



Retropropagação – um exemplo (14/14)

• Resultado após a iteração

- RN com seus novos parâmetros
- A RN executará diversas iterações por várias épocas até o fim do treino



W		
0.192	0.4	-0.508
-0.306	0.1	0.194

$b^{[1]}$
-0.408
0.194

U	
-0.261	-0.138

$b^{[2]}$
0.218

IMPORTANTE

- **O exemplo mostrado refere-se a:**
 - Uma RNFF para classificação binária.
 - Com 1 camada escondida.
 - Com a unidade de saída usando sigmoide como função de ativação.
 - Com as unidades escondidas usando sigmoide como função de ativação.
 - Adotando a SQE como função de custo.
- Se as configurações mudarem (ex.: outra função de ativação for adotada), o processo é até parecido...
- ... porém, as derivadas das funções mudam e, por consequência, os cálculos ficam diferentes.

Notebook

- No notebook abaixo, você poderá acompanhar a perda da RNFF durante várias épocas:
- <https://colab.research.google.com/drive/1UV33SK9FPu4RdApvRXwbXk3sbaEYqMqR?usp=sharing>

Execute as células de código referentes às etapas

1. Importa a base de dados de treinamento
2. Separação das partes X e Y
3. Pré-processamento da Base de Dados
7. Visualizando a convergência do modelo

Ou seja, execute 1, 2 e 3 e depois pule para a 7

Referências

- JURAFSKY, D.; MARTIN, J. H. Neural Networks. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 12, 2025 draft). [s.l.]: [s.n.], 2025. cap 7. Disponível em: < <https://web.stanford.edu/~jurafsky/slp3/> >. Acesso em: 22 abr. 2025.
- HAN, J.; KAMBER, M.; PEI, J. Classification by Backpropagation. *In*: HAN, J.; KAMBER, M.; PEI, J. **Data Mining: Concepts and Techniques**. 3. ed. Waltham: Morgan Kaufmann, 2011. cap 9, p. 398-408.
- DE OLIVEIRA, M. A.; MELO JUNIOR, F. A. P. **Backpropagation e Redes Neurais**. Rio de Janeiro: Ed. Ciência Moderna, 2024.